

第七类第 15、16 题：手工推导与程序实现讲解

阶段二学习稿 | 2026-09-09

本文的目标不是只给出程序，而是把“为什么这样离散、手算时每一步是什么、程序怎样重复这些步骤”连起来。文中的示例计算采用较小网格，便于手算；正式实验应由 MATLAB 输出误差、收敛阶和图表。

1. 先建立整体认识

数值分析处理的核心困难是：微分方程通常无法在每一点直接写出数值答案。计算机也不能处理“连续的无穷多个点”，所以要先把连续问题转换为有限个代数计算，这一步叫**离散化**。

- **有限差分法**：把导数换成相邻网格点的差商，得到线性方程组。它适合边值问题，例如第 15 题。
- **Runge-Kutta 法**：在每个小区间内多次试探斜率，再加权平均推进。它适合初值问题，例如第 16 题。
- **打靶法**：把边值问题暂时猜成初值问题，调整未知初始量，使积分到终点时满足另一端边界条件。
- **符号计算**：用代数方法求解析表达式，作为数值解的参照，不等于数值算法本身。

手算与 MATLAB 的数学步骤相同；区别在于手算只做少量节点，程序用循环自动处理许多节点，并负责误差、绘图和重复实验。

2. 第 15 题：有限差分法

2.1 连续问题

$$u''(x) = e^{-x^2}, \quad -1 \leq x \leq 1, \quad u(-1) = u(1) = 0.$$

把区间等分为 N 段：

$$h = \frac{2}{N}, \quad x_i = -1 + ih, \quad u_i \approx u(x_i).$$

2.2 导数怎样变成差分

在 x_i 附近展开：

$$u(x_i + h) = u_i + hu'_i + \frac{h^2}{2}u''_i + \frac{h^3}{6}u'''_i + \cdots,$$

$$u(x_i - h) = u_i - hu'_i + \frac{h^2}{2}u''_i - \frac{h^3}{6}u'''_i + \cdots.$$

两式相加，奇数项抵消：

$$u(x_i - h) - 2u(x_i) + u(x_i + h) = h^2u''(x_i) + O(h^4).$$

因此

$$\boxed{\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} = e^{-x_i^2}}.$$

这是二阶中心差分，截断误差为 $O(h^2)$ 。这里“二阶”表示网格步长缩小一半时，主要误差大约缩小四倍。

2.3 手算一个小网格: 取 $N = 4$

此时 $h = 2/4 = 0.5$, 节点为

$$x_0 = -1, x_1 = -0.5, x_2 = 0, x_3 = 0.5, x_4 = 1.$$

边界值是 $u_0 = u_4 = 0$ 。内点方程为

$$\frac{u_0 - 2u_1 + u_2}{0.25} = e^{-0.25},$$

$$\frac{u_1 - 2u_2 + u_3}{0.25} = 1,$$

$$\frac{u_2 - 2u_3 + u_4}{0.25} = e^{-0.25}.$$

乘以 0.25, 得到

$$-2u_1 + u_2 = 0.25e^{-0.25},$$

$$u_1 - 2u_2 + u_3 = 0.25,$$

$$u_2 - 2u_3 = -0.25e^{-0.25}.$$

令 $a = 0.25e^{-0.25}$ 。由于题目关于 $x = 0$ 对称, 可设 $u_1 = u_3 = p$ 、 $u_2 = q$, 于是只需解

$$-2p + q = a, \quad 2p - 2q = 0.25.$$

第二式给出 $p - q = 0.125$, 即 $p = q + 0.125$ 。代入第一式:

$$-2(q + 0.125) + q = a \Rightarrow q = -a - 0.25,$$

$$p = -a - 0.125.$$

这就是一个完整的“手算数值解”: 先列网格, 再列差分方程, 最后解三元方程组。实际计算时 a 再用计算器或 MATLAB 求其小数。由于本题右端为正而算出的解为负, 也符合 $u'' > 0$ 且两端为零时曲线向下弯的形状。

2.4 矩阵写法与程序对应

一般 N 下, 未知向量为

$$\mathbf{u} = (u_1, \dots, u_{N-1})^T,$$

矩阵为

$$A = \frac{1}{h^2} \begin{bmatrix} -2 & 1 & & \\ 1 & -2 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & -2 \end{bmatrix}, \quad b_i = e^{-x_i^2}.$$

程序的思路就是:

1. 生成 x 和 h ;
2. 取内部节点 $x(2:N)$;
3. 生成三对角矩阵 A ;
4. 计算 $b = \exp(-xInner.^2)$;
5. 用 $uInner = A \setminus b$ 解线性方程组;
6. 把两个边界零值拼回完整向量。

不应写 $inv(A)*b$, 因为 $A \setminus b$ 更稳定、更快, 也直接体现“解方程组”的数学含义。

2.5 为什么还要解析解和误差

两次积分可得本题解析解

$$u(x) = \frac{1}{2}e^{-x^2} + \frac{\sqrt{\pi}}{2}x \operatorname{erf}(x) - \frac{1}{2}e^{-1} - \frac{\sqrt{\pi}}{2} \operatorname{erf}(1).$$

程序在同一组节点计算 u_{exact} , 再求

$$E_{\infty} = \max_i |u_i - u_{exact}(x_i)|.$$

用 $N, 2N, 4N$ 重复计算, 观测阶写成

$$p \approx \log_2 \left(\frac{E_N}{E_{2N}} \right).$$

若中心差分实现正确, p 应接近 2。这个数必须由程序实际运行得到, 不能凭经验填入报告。

3. 第 16 题: 经典四阶 Runge-Kutta 法

3.1 连续问题与解析解

$$y' = -y + t^2 + 3, \quad y(0) = 1.$$

改写为 $y' + y = t^2 + 3$ 。设特解为 $at^2 + bt + c$, 代入得

$$2at + b + at^2 + bt + c = t^2 + 3.$$

比较系数: $a = 1, 2a + b = 0, b + c = 3$, 所以 $b = -2, c = 5$ 。通解为

$$y(t) = t^2 - 2t + 5 + Ce^{-t}.$$

由 $y(0) = 1$ 得 $C = -4$, 故

$$y_{exact}(t) = t^2 - 2t + 5 - 4e^{-t}.$$

3.2 RK4 每一步到底做什么

设 $t_{n+1} = t_n + h$, 令 $f(t, y) = -y + t^2 + 3$ 。经典 RK4 为

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + h/2, y_n + hk_1/2), \\ k_3 &= f(t_n + h/2, y_n + hk_2/2), \\ k_4 &= f(t_n + h, y_n + hk_3), \end{aligned}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

这里的 k_i 是斜率, 不包含 h 。常见错误是把 h 乘进 k_i 后又在最后乘一次 h , 这样会重复乘步长。

3.3 手算第一步: 取 $h = 1$

从 $t_0 = 0, y_0 = 1$ 出发:

$$k_1 = f(0, 1) = -1 + 0 + 3 = 2.$$

第二个试探点: $t = 0.5, y = 1 + 2/2 = 2$, 所以

$$k_2 = f(0.5, 2) = -2 + 0.25 + 3 = 1.25.$$

第三个试探点: $t = 0.5, y = 1 + 1.25/2 = 1.625$, 所以

$$k_3 = f(0.5, 1.625) = -1.625 + 0.25 + 3 = 1.625.$$

第四个试探点: $t = 1, y = 1 + 1.625 = 2.625$, 所以

$$k_4 = f(1, 2.625) = -2.625 + 1 + 3 = 1.375.$$

加权平均:

$$y_1 = 1 + \frac{1}{6}(2 + 2 \times 1.25 + 2 \times 1.625 + 1.375).$$

因此, 手算过程已经完全确定; 计算器只负责最后的小数运算。接下来从 (t_1, y_1) 重复同样的四次取斜率, 就得到 y_2, y_3 。

3.4 为什么是四阶

RK4 对单个步长的局部误差为 $O(h^5)$, 累计到固定区间后全局误差为 $O(h^4)$ 。所以用 $h, h/2$ 两次计算时, 误差比应大致满足

$$E(h)/E(h/2) \approx 2^4 = 16.$$

同样, 实际比值和收敛阶必须来自 MATLAB 输出。

3.5 计算机实现思路

rk4Solve 函数只做一件事: 给定函数句柄、区间、初值和步数, 循环执行上面的四个斜率计算。主脚本负责:

- 选择 N 或 h ;
- 调用求解器;
- 计算解析解;
- 计算最大误差和收敛阶;
- 绘制数值解、解析解和误差曲线。

这就是“手算公式”与“程序循环”的——对应关系: 手算的一步变成循环体, 手算的下一步变成下一次循环。

4. 打靶法是什么，以及它和本项目的联系

打靶法通常用于边值问题。假设

$$u'' = f(x), \quad u(-1) = 0, \quad u(1) = 0.$$

左端的函数值已知，但初始斜率 $s = u'(-1)$ 未知。先猜一个 s ，把问题改写为

$$u' = v, \quad v' = f(x), \quad u(-1) = 0, \quad v(-1) = s.$$

用 RK4 从 -1 积分到 1 ，得到终点 $u(1; s)$ 。如果不等于 0 ，就改变 s ，直到满足右边界。这像“向靶子射击”，所以叫打靶法。

本题是线性的，因此可以用两次积分直接确定 s ：分别取 $s = 0$ 和 $s = 1$ ，由线性关系插值得到使 $u(1; s) = 0$ 的斜率。非线性问题一般需要割线法或牛顿法迭代。

有限差分法直接同时求全部网格点；打靶法沿区间推进。前者要解线性方程组，后者要反复解初值问题。两者都需要离散化，只是离散位置不同。

5. 手算、MATLAB 和离散化的区别

手算时只能选择很小的 N ，逐个写出节点和中间量；MATLAB 可以使用几十、几百甚至更多节点，并自动重复计算。数学公式没有改变，改变的是：

- 连续区间变成有限网格；
- 导数变成差分或斜率组合；
- 无限过程变成有限循环；
- “看起来接近”变成最大误差、收敛阶和图形验证。

对于第 15 题，离散化直接产生代数方程组；对于第 16 题，离散化表现为时间步进。二者都要做步长实验，否则无法证明算法确实达到理论阶数。

6. 建议的 MATLAB 程序结构

```
src/
  poisson/
    solvePoissonFD.m      % 生成网格、组装三对角矩阵并求解
ode/
  rk4Solve.m             % 单独实现 RK4
common/
  errorNorms.m           % 统一计算误差
  computeOrder.m         % 根据误差序列计算收敛阶
tests/
  runAllTests.m          % 检查公共函数和基本结果
```

建议的调用关系是：入口脚本设置参数，调用算法函数，保存 `results/*.mat`，再绘图。算法函数不负责画图，绘图脚本也不应重新实现差分或 RK4。这样答辩时可以清楚说明每个文件的职责。

7. 最后应怎样验证

1. 第 15 题：用解析解计算 E_∞ ，并用 $N, 2N, 4N$ 验证二阶收敛；可再用制造解检验差分格式。
2. 第 16 题：用解析解计算误差，并用不同步长验证四阶收敛；检查 k_i 是否采用同一种约定。
3. 两题都要检查边界点、初值点和向量长度。
4. 报告中的具体误差、收敛阶、表格和图都应从程序实际输出取得。

这样，阶段二的“手工推导”就完成了三个目标：知道公式从哪里来，能够亲手完成一个小例子，也知道程序如何把同一过程扩展到大量网格点。